

ConnectBox Software Specification

Oct 7, 2026 · @Kirk Wilson

ConnectBox software turns a small single-board computer into an offline Wi-Fi library: it runs its own hotspot and captive portal, indexes a USB drive into multilingual menus, and serves video, audio, documents and offline websites to any phone browser with no internet. This specification describes what the code in connectbox-pi and its sibling repositories does, as of the current master branch.

1. Scope, users and constraints

A ConnectBox serves one place at a time (a clinic, school, church or village) where there is no reliable internet. People join its Wi-Fi with the phone they already own and browse a media library in their own language.

Role	What they do
Visitor	Joins the Wi-Fi (default name "Connectbox - Free Media"), is sent to the menu by the captive portal, watches, reads, downloads, chats
Content preparer	Copies media onto a USB drive in language folders; the box builds the menus from it
Administrator	Changes Wi-Fi name, password, branding and settings in the admin pages; reads usage statistics
Organization	Optionally runs a central server that collects statistics and sends content packages to many boxes

Hardware: NanoPi NEO (main test unit), Raspberry Pi 3/4 and CM4, Orange Pi Zero 2, Radxa CM3. The model is read from `/sys/firmware/devicetree/base/model` at provisioning time. Boards run Armbian or Raspberry Pi OS (Debian Buster/Bullseye/Bookworm family).

Design constraints:

- Works with no internet at all; internet, when present, is used only for optional extras (online translation lookups, central server sync).
- Must work on very old phones (iOS 9, Android 4–5 era browsers): plain pages, no app install.
- Small boards: 512 MB RAM on the NEO, so conversions stream and only Python-standard or pure-Python libraries are used on the box.
- The Wi-Fi MAC address changes on every boot by design, so the box's address on an upstream network is never fixed.
- One USB drive holds all content; FAT32, exFAT, NTFS and Linux file systems (ext4 and others) are supported.

2. Repositories and components

connectbox-pi is the integration point: its Ansible playbook installs everything else onto the board. All repositories are on GitHub under the ConnectBox organization (Kirk's fork mirrors connectbox-pi).

Repository	Runs on	What it provides
connectbox-pi	Box	Ansible playbook and roles; <code>mmiLoader.py</code> (USB indexer); <code>PxUSBm.py</code> (USB mounter and network recovery); nginx sites; scripts that patch the compiled app; translations; tests
connectbox-mediainterface	Box (browser)	The Angular/Ionic media app visitors use; its GitHub Actions publish a latest release on every push to main. Also ships <code>lazyLoader.py</code> for OpenWell packages
connectbox-admin-ui	Box (browser)	The admin pages at <code>/admin</code>
connectbox-manage	Box	Node.js backend on port 5002 behind the admin pages and the <code>connectboxmanage</code> command
connectbox-client	Box	Classic (non-enhanced) web interface, cloned by the <code>webserver-content</code> role
simple-offline-captive-portal	Box	Flask captive portal on port 5001
connectbox-wifi-configurator	Box	Helper that writes hostapd Wi-Fi settings
connectbox-hat-service	Box	OLED display, buttons, battery level and safe shutdown
connectbox-chathost	Server	Central server: statistics, chat relay, OpenWell package links, MediaBuilder (Bolt CMS)
connectbox-tools	Build Pi	Scripts that provision a board and turn it into a release image
connectbox-reports, connectbox-access-log-analyzer	Server or PC	Usage reporting from logs

Repository	Runs on	What it provides
armbian-build	Build machine	Board operating system images

3. Provisioning

A box is built by running one Ansible playbook against a freshly flashed board: `cd ansible && ansible-playbook -i <device_ip>, site.yml -e wireless_country_code=US`. `site.yml` loads variables for the OS family, distribution and board, then runs the `connectbox-pi` role, whose dependencies install the system in this order:

Order	Role	What it installs or changes
1	bootstrap	Core files: <code>mmiLoader.py</code> , <code>PxUSBm.py</code> , <code>ConnectBoxManage.sh</code> , shutdown scripts, <code>brand.j2</code> , translations, <code>logrotate</code> ; Python libraries (mammoth, <code>openpyxl</code> , <code>xlrd</code>)
2	hat-service	Larger <code>/tmp</code> and swap, OLED and battery service (<code>neo-battery-shutdown</code>)
3	dns-dhcp	<code>dnsmasq</code> (DHCP + DNS that answers every name with the box) and <code>avahi</code>
4	network-interfaces	Removes <code>NetworkManager</code> and <code>systemd-resolved</code> ; phone tethering tools; interface files
5	wifi-ap	<code>hostapd</code> access point and the Wi-Fi configurator
6	<code>mikegleasonjr.firewall</code>	<code>iptables</code> : allow SSH, DHCP, DNS, mDNS, HTTP and HTTPS in
7	enhanced-content	Downloads the latest media app and admin UI releases, <code>ffmpeg</code> , icons, <code>types.json</code> additions, and runs the <code>patch_*.py</code> scripts on the app
8	nginx	Five sites (see section 4), home button snippet, <code>gzip</code> , caching rules

Order	Role	What it installs or changes
9	kiwix	kiwix-tools 3.8.2 for the board's CPU, kiwix-serve service, empty library
10	captive-portal	Flask portal in its own virtualenv, systemd unit
11	webserver-content	Classic interface; Flask + gunicorn chat/admin API on port 5000
12	usb-content	systemd-udev settings; removes the retired udev automount rule
13	sample-content, rpi-permissions, wlan-set, remove-lz4	Mount point, permissions, final Wi-Fi settings, cleanup
last	image-preparation (optional)	Clears per-box state (admin session key, logs, Wi-Fi regulatory domain) before an image is made

The hardware-only roles are skipped for an AWS build (`aws_instance`). Device settings live as JSON in `/usr/local/connectbox/brand.j2` (the single source; `brand.txt` was retired), read and written with `jq` by `ConnectBoxManage.sh` and directly by `connectbox-manage`.

4. Network and web serving

The box is its own small network: `hostapd` runs the access point, `dnsmasq` hands out addresses on `10.129.0.1` and answers every DNS name with the box, and `nginx` on port 80 sends each request to the right service. Only `nginx`, `SSH`, `DHCP`, `DNS` and `mDNS` are reachable from outside; every backend listens on `127.0.0.1`.

Captive portal. Phones probe a known address when they join Wi-Fi (`/generate_204`, `/hotspot-detect.html`, `/ncsi.txt`, `/connecttest.txt`, `/.well-known/captive-portal` and others). `nginx`'s default site passes these to the Flask portal (port 5001), which answers each phone family in the way that makes it open a sign-in window showing the ConnectBox menu. The portal keeps per-client session timing so a rejoin is recognised, and keeps special handling for old iOS and Android versions on purpose. Typing `gowifi.org`, `wi.fi` or `go` also reaches it.

Upstream connection (optional). A second Wi-Fi interface (client mode), Ethernet, or a tethered phone over USB can give the box internet. `dhcpcd` must not manage the client Wi-Fi interface (`denyinterfaces` placed before any interface block), otherwise the box takes two addresses. `PxUSBm` restarts `hostapd` and checks interfaces if the access point drops.

Port	Service	Reached at
80	nginx (enhanced site for connectbox, .local; captive-portal default site; classic, icon-only and static sites when chosen)	Everything below
5001	Captive portal (Flask + gunicorn)	Captive-portal probes
5002	connectbox-manage (Node.js, PM2)	/admin/api, /chat
5000	Flask chat and admin API (python/main.py, gunicorn)	Classic interface
8090	kiwix-serve	/kiwix/ (ZIM files)

The enhanced site (connectbox_enhanced.conf.j2) serves the media app from /var/www/enhanced/content/www, the admin UI at /admin, USB files at /content/ and adds these behaviours:

- / and /index.html are rewritten to /tmp/connectbox-indexing.html while a full index runs, and are always sent with Cache-Control: no-cache.
- A floating home button is inserted before </body> of every web-content and ZIM page (subs_filter, needs libnginx-mod-http-subs-filter).
- A 404 under /assets/content/<lang>/html/ returns to the start page of the item the link was clicked in (from the Referer), not a blank page.
- /kiwix/ is proxied with ^~ so the .json rule never takes ZIM files; Kiwix HTML is sent no-cache because nginx changes it.
- /build/ and /admin are sent no-cache so app updates reach browsers; text assets are gzipped.

5. USB content pipeline

Plugging in a USB drive is the normal way to load content: PxUSBm mounts it and starts mmiLoader, which either restores the saved menus in seconds or indexes the whole drive (minutes on a large drive). Removing the drive clears the menus.

1. **Mount (PxUSBm.py).** A systemd service polls lsblk about every 3 seconds. A new drive is mounted at /media/usb0: its first partition (sda1) or a whole disk with a file system and no partition table. FAT drives get dosfsck and the utf8 option; Linux file systems skip both. PxUSBm is the only mounter: a udev mounter was retired because the two raced.
2. **Start the loader.** On its next poll PxUSBm starts mmiLoader.py in the transient systemd unit connectbox-loader; the marker /tmp/.usb0_indexed stops a second start. mmiLoader itself refuses to run twice (marker /tmp/creating_menus.txt).
3. **Prepare.** On Linux file systems mmiLoader adds read permission for everyone where it is missing (like chmod -R a+rX, symlinks skipped) so nginx can read files written by another computer's user. It rebuilds the Kiwix library from the drive's .zim files, then removes the old menus.

4. **Fast path: saved.zip.** If content/saved.zip (or one at the USB root) exists, it is extracted into /var/www/enhanced/content/www/assets/content/, translations and right-to-left flags are refreshed, and the run ends.
5. **Full index.** Otherwise mmiLoader writes a "Loading new content" page (/tmp/connectbox-indexing.html, progress "N / M" in every language on the drive, refreshing itself), walks each language folder, builds cards and JSON (section 6), extracts thumbnails with ffmpeg, removes the loading page and writes saved.zip last. It checks the drive is still mounted before each folder and before writing saved.zip, so a pulled drive never leaves a broken cache.
6. **Remove.** When the drive disappears PxUSBm stops the loader, empties the Kiwix library, runs mmiLoader.py --clear (empty English menu, default languages) and lazily unmounts.

Deleting saved.zip from the drive forces a full index on the next insert; the admin's "Load Content From USB Flash" does the same. Thumbnails are cached on the drive as hidden .thumbnail-<lang>-<slug>.png files, so re-indexing is faster than the first run.

6. Content model

mmiLoader turns folders and files into cards, each described by JSON the media app reads: per language, <lang>/data/main.json (the card list), one data file per card, types.json (extension map), interface.json (menu wording) and a shared languages.json. Media are linked into a flat <lang>/media/ folder; converted pages go to <lang>/html/<slug>/. The full layout rules are in the USB Content Structure Reference.

Folder types (decided per folder, in priority order):

Folder	Rule	Becomes
Web content	Has index.html / index.htm	One web card opening that site; optionally also a downloadable zip
Android app	Has AndroidManifest.xml	One app download card
Complex	Has subfolders at any depth	A generated file-browser web page (indexer.py)
Collection	More than 2 files, no subfolders	One card with an episode list
Singular	1 or 2 files	One card per file
Language root	en/, es/, zh-CN/ ...	Not a card; its loose files are singular cards

Card pictures come from folder art (folder.png, folder.jpg, cover.jpg, album.art.jpg, front.jpg, then any picture), a video frame from ffmpeg (seeking past black frames), cover art inside audio files, the picture itself, or an icon for the file type. Hidden control files in content/ change behaviour: .language (language of a drive with no language folders), .compress, .NoWebcompress.

File types. `types.json` maps about 55 extensions to a media type, MIME type and icon; files with other extensions, or names starting with `.` or `_`, are skipped. Videos must be H.264 in MP4 to play in browsers. Two files with the same name in different folders share a link only if they are identical copies; otherwise the later one is linked as `<name>--<folder><ext>` (`unique_media_name`), keeping slugs and statistics stable.

Office files shown as web pages. `make_document_web_page()` converts these at index time into `<lang>/html/<slug>/index.html`; the card becomes a web card but keeps the file's MIME type, so the app shows a details page with an open (book) button and a download of the original. A failed conversion leaves a download-only card.

Type	Library	What the page shows	Not kept
Word <code>.docx</code>	mammoth 1.13.0	Text, headings, lists, tables, pictures, links	Headers, footers, text boxes, columns
Excel <code>.xlsx</code> / <code>.xls</code>	openpyxl 3.1.5 (streaming) / xlrd 2.0.1	One table per visible sheet, saved formula results, first 2,000 rows and 100 columns, sheet links	Charts, colours, number formats
PowerPoint <code>.pptx</code>	Python standard library only	One numbered section per visible slide: title, bullets with levels, text boxes, pictures, tables, speaker notes	Slide design, animations, charts, diagrams

`.doc`, `.ppt` and other formats stay download-only. Pages are full HTML with right-to-left direction for RTL languages.

ZIM files (offline websites) are served by kiwix-serve. Each `.zim` becomes its own web card, even inside a collection, with title, description and icon from the Kiwix library (unreadable or invisible icons replaced by a web icon, dark ones placed on white). Its page redirects to `/kiwix/content/<name>/`. A ZIM tagged with several languages is also listed in the box's other matching languages; TED ZIMs open in the menu's language. Kiwix's own search works inside each ZIM.

7. User interface

Visitors use the enhanced interface: the `connectbox-mediainterface` app (Angular/Ionic), downloaded as its latest GitHub release and then changed on the box by idempotent `patch_*.py` scripts and inline shell patches in the enhanced-content role. Changing the app this way keeps boxes working with the published release while ConnectBox-specific fixes are made.

What a visitor sees. A home page of cards grouped by category and "For You", a Language button, search, collections with episode lists, a details page per item with open and download buttons (the download button shows the file size), built-in players and PDF/EPUB readers, a chat page, and a

Configuration link to the admin pages. Web content and ZIM pages carry a home button back to the menu.

Patch	Bundle	Behaviour
goToDetails	6.js	Web cards open straight away (same tab) and report the view to /admin/api/weblog themselves; Word, spreadsheet and presentation cards go to the details page like PDFs
patch_zim_download.py	3.js	Details page: ZIM cards have no download; Office cards download the original from media/<file> and open with the book icon in the same tab
patch_download_size.py	0, 1, 3, 4, 5, 7.js	File size next to every download button, from a HEAD request; nothing shown if unknown
loadData language fix	3.js	On a language change, return to the root page first (prevents "media missing!")
patch_language_button.py	6.js	Language button shows the active language's own name
patch_footer_translation.py	6.js	Configuration link in the menu's language
patch_chat_rtl.py	2.js, main.css	Translated chat labels, per-message text direction, bubble placement
patch_saved_language.py	main.js	A returning visitor's saved language is refreshed from the current languages.json
patch_browser_language.py	main.js	First-time visitors start in the first browser language the box has; a language picked in the menu always wins

Each script exits non-zero when the text it expects is missing. Because every push to mediainterface main publishes a new release, all patches must be re-run against a new build before boxes take it.

The classic, icon-only and static-site interfaces remain in the playbook as alternatives (`enhanced_interface: False`).

8. Languages

Each top-level folder on the drive named with a language code (`en`, `es`, `ar`, `zh-CN`, or 3-letter codes such as `spa`) becomes a language in the menu; a drive with no language folders is English unless `content/.language` says otherwise. Codes are normalised with `languageCodes.json` (`per` → `fa`, `spa` → `es`), and the same mapping places multi-language ZIM files.

- **Menu wording.** The app only has English, so `mmiLoader` writes each language's `interface.json` from `/usr/local/connectbox/translations/<code>.json`. Reviewed files ship for Farsi, Arabic, Spanish, Portuguese and Chinese. Any other language is translated online once (MyMemory) when the box has internet and saved there; offline it stays English and is retried next time. Each entry stores the English it came from, so hand corrections stick until the English changes. The same strings serve pages `mmiLoader` writes itself (loading page, "Speaker notes").
- **Right-to-left.** Arabic, Farsi, Hebrew, Urdu, Pashto, Sindhi, Kurdish (Sorani), Uyghur, Yiddish and Dhivehi get `"rtl": true` in `languages.json`; the app, converted pages and chat flip direction.
- **Language button** shows each language's own name from `languageCodes.json`.
- **Not translated:** content titles (file names), the brand name, chat messages.

9. Administration, statistics, chat and display

The admin pages (`/admin`, `connectbox-admin-ui`) talk to `connectbox-manage` on port 5002, which reads and writes `brand.j2` and runs system commands; the same calls are available on the box as `connectboxmanage get|set|do ...` (which fails with a traceback if port 5002 is down).

- **Login:** password-protected; the session cookie is signed with a random key created on the box on first start (`/usr/local/connectbox/admin_session.key`, root only), never a key in the source.
- **Settings:** admin password, Wi-Fi name (SSID), channel, optional WPA passphrase, country code, host name, branding (name, logo, intro text), upstream Wi-Fi client, central server address and authorisation.
- **Actions:** reload content from USB (deletes `saved.zip` and re-indexes), load or refresh OpenWell packages, sync with the server (`phonehome.py`), view logs, reboot, shut down, wipe.
- **Statistics:** the app reports each view or download with `PUT /admin/api/weblog`; the admin pages show the top items per hour, day, week, month and year and let them be downloaded or cleared. Statistics can be turned off (`disable_stats`).
- **Chat:** `PUT /chat` stores messages that every visitor sees; each message carries its sender's text direction.
- **OLED display and buttons** (`connectbox-hat-service`, `neo-battery-shutdown` service): status pages with battery level, connected users, IP address and the most-used files per period; a "creating menus" message while indexing; holding the left button 4 seconds or more shuts the box down safely (AXP209 power chip set over `i2c`, then `shutdown -h`).

10. Central server and OpenWell

A box works alone; an organization can also run a central server (connectbox-chathost) that many boxes report to when they get internet.

- **Sync.** When the box is online (tethered phone or upstream Wi-Fi) `phonehome.py` sends its usage statistics and chat messages to the server named in the admin pages and fetches what is waiting for it.
- **OpenWell content packages.** The admin's Web Server section lists packages from the server (`get.subscriptions`); subscribing writes `assets/content/subscription.json`. Nothing downloads until the admin presses "Download Missing Content Files", which runs `lazyLoader.py` (from `mediainterface`) to fetch the package's menus and media. Packages are built on the server with MediaBuilder (Bolt CMS), linked at `/chathost/link/openwell`.
- **Server build.** The chathost playbook targets Ubuntu 24.04: Docker with compose v2, nginx with Let's Encrypt HTTPS, data under `/srv/connectbox`, secrets in an untracked `secrets.yml`, application and database ports bound to 127.0.0.1. It has been checked (syntax, nginx config, dependency install) but not yet run on a real server.

11. Security, reliability rules and invariants

These rules come from failures seen on real boxes; changes must keep them true.

Rule	Why
Backends listen on 127.0.0.1 only; the firewall admits SSH, DHCP, DNS, mDNS, HTTP, HTTPS	Visitors reach services only through nginx
Admin session key is per box and deleted when an image is prepared	The old fixed key was public and allowed forged admin logins
PxUSBm is the only USB mounter and owns network recovery	A second mounter and a separate watchdog raced it
<code>denyinterfaces</code> before any <code>interface</code> block in <code>dhcpcd.conf</code> ; never reconfigure the client Wi-Fi over SSH on that link	Otherwise two addresses, or the box drops off the network
<code>saved.zip</code> is written last, after checking the drive is still mounted	A pulled drive must not leave a broken cache
The loading page is keyed on <code>/tmp/connectbox-indexing.html</code> , not <code>creating_menus.txt</code>	The OLED buttons also write that file
Scripts use Unix line endings (<code>.gitattributes</code>)	A Windows line ending makes a script fail silently on the box
<code>brand.j2</code> must stay valid JSON	Several programs parse it
<code>mimeType</code> in card JSON is the real MIME type	Browsers decide playback from it

Rule	Why
(video/mp4)	
Captive-portal checks for old iOS and Android stay	Deployed phones are that old
The Wi-Fi MAC (and so the upstream address) changes every boot	Intentional; do not pin it

Known risks: the captive portal pins old library versions (Flask 1.1.1, gunicorn 20.0.4) with open advisories, kept for old-phone compatibility until tested; the patch scripts depend on exact compiled text and must be re-checked for each app release.

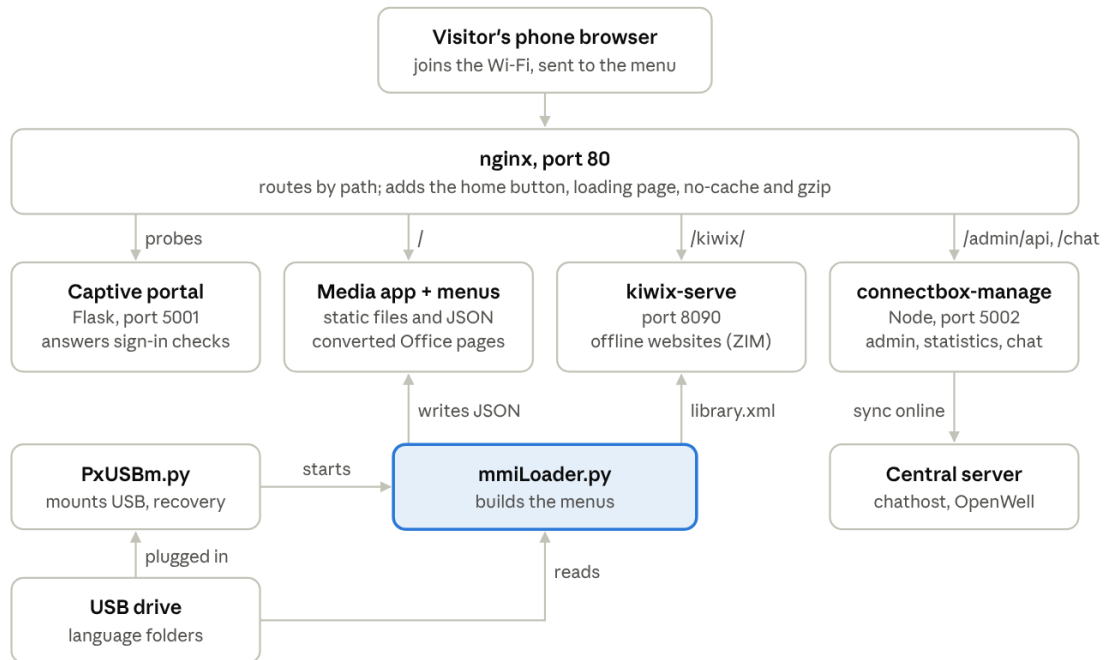
12. Testing, images and releases

Activity	How
mmiLoader tests	<code>ansible/roles/bootstrap/files/test_mmiLoader.py</code> , 264 checks in 20 scenarios (folder types, translations, ZIM, Word, spreadsheets, presentations, icons, duplicate names); run after every mmiLoader change; kept Python 3.7-compatible
USB mount tests	<code>test_PxUSBm_mount.py</code> simulates <code>PxUSBm.mountCheck()</code> with fake <code>lsblk</code> output
On-device check	Copy the changed file to the test unit (strip Windows line endings), force a re-index, check in a phone browser; patch scripts are run against the real release bundles
Lint	<code>ansible-lint ansible/site.yml</code>
Release image	<code>connectbox-tools</code> on the build Pi: <code>make_cb.py</code> provisions a board, <code>prepare_release.py</code> runs <code>image-preparation</code> and writes the image to <code>deployment/Images</code> ; images are published on the connectbox-pi releases page (see <code>docs/making_an_image.md</code>)
App release	Every push to <code>connectbox-mediainterface</code> <code>main</code> builds and publishes <code>latest</code> , which new boxes download

Work happens on master in connectbox-pi with conventional commit messages, pushed to both the ConnectBox organization and Kirk's fork. Code comments follow the repository's policy: every function, loop and section explains what it does and why.

13. Architecture at a glance

Visitors reach every service through nginx; the USB drive feeds the menus



ConnectBox components on the box and the central server

Phones only ever talk to nginx; the USB drive reaches them through PxUSBm and mmiLoader, which write the menus and the Kiwix library. The OLED display and buttons (hat service) run alongside and read the same statistics.